

Unsteady Streamline Synthesis

Chen-Yuan Hsu
Bournemouth University
United Kingdom
cyhsu.bu@gmail.com

Li-Yi Wei
Adobe Research
United States of America
liyiwei@acm.org

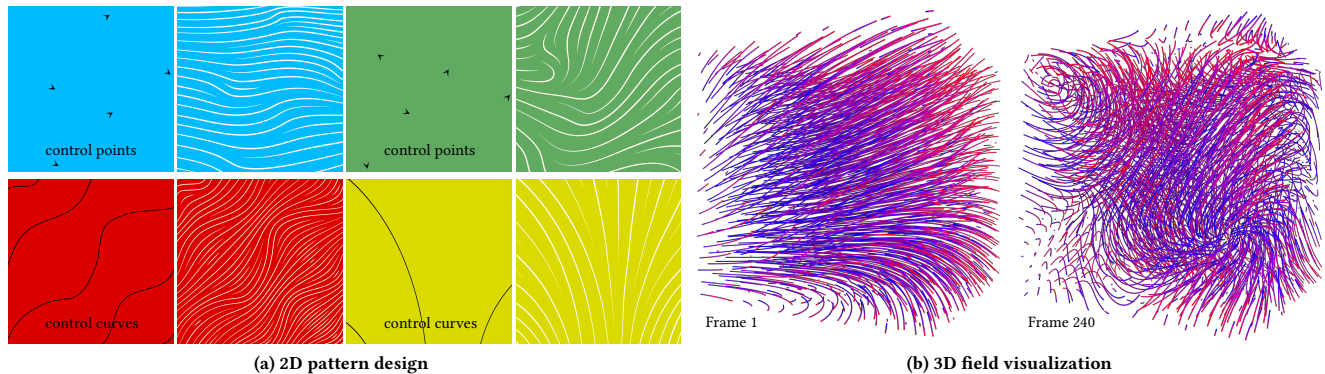


Figure 1: Time-varying streamline arrangements. Our framework can not only synthetically organize a group of streamlines following partial control points/curves (a) or full turbulence fields (b) under 2D (a) and 3D (b) time-varying scenarios but also controllably customize stylized (a) or adaptive (b) streamline arrangements via extra scalar inputs (a) or local flow similarity (b)

Abstract

Streamlines tracing out a vector field are important for graphics and visualization, yet the way of controllably arranging the streamlines in unsteady flows, even in 2D, has not been well studied to this day. We thus propose an *unsteady streamline synthesis* framework that can create a full set of coherent streamlines following certain inputs under 2D and 3D time-varying scenarios while satisfying coverage, continuity, and controllability purposes. Our key concept is to treat the time-varying streamline arrangement like a dynamic packing of anisotropic elements with deformability such that both quality and efficiency can be fulfilled. Specifically, we devise a synthesis process based on inter- and intra-streamline relationships to dynamically synthesize all the coherent streamlines in parallel and incorporate a regulation mechanism with primitive operations to timely adjust each streamline's continuity and the overall streamline coverage. With specification, our framework can further controllably produce desired streamline arrangements for various application demands.

CCS Concepts

• Computing methodologies → Computer graphics.

Keywords

element synthesis, streamline placement, vector field visualization

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1140-4/24/12

<https://doi.org/10.1145/3681758.3697981>

ACM Reference Format:

Chen-Yuan Hsu and Li-Yi Wei. 2024. Unsteady Streamline Synthesis. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3697981>

1 Introduction

Streamline synthesis plays a vital role in graphics and visualization. However, as reported in previous surveys [McLoughlin et al. 2010; Sane et al. 2020], methodically orienting individual streamlines in unsteady flows (i.e. time-dependent vector fields) remains unsolved.

A streamline is an integral curve whose direction at each point is tangent to the direction of an underlying vector field at that point. For the creation of well-distributed streamlines, a majority of means (e.g. [Liu et al. 2006]) adopt a seeding strategy to gradually calculate integral curves following given vectors and some other approaches (e.g. [Turk and Banks 1996]) concurrently marshal each streamline via an optimization procedure. Nevertheless, most of them focus on time-independent scenarios, as there exists a technical difficulty in avoiding temporal aliasing or incoherence artifacts emerging from animations. Even if a few methods (e.g. [Jobard and Lefer 2000]) can animate the correlated streamlines, owing to methodological curbs, they merely function for 2D cases and streamline controllability is not well handled. We notice that the ideal solution that can properly generate controllable streamline arrangements in unsteady flows, particularly in 3D, has been a challenging and open problem so far.

To address this crucial gap, we propose a universal framework of *unsteady streamline synthesis* with the capability of synthetically constructing a complete pack of coherent streamlines, in possession of satisfactory coverage, continuity, and controllability, conforming to partially or fully specified inputs (e.g. control points/curves or simulated velocity fields) under 2D and 3D time-varying scenarios.

Since a streamline is like an elongated element with adjustable length and shape, the main idea behind our framework is to consider the overall streamline arrangement under the time-varying status as an anisotropic element packing pursuant to dynamic orientations. As inspired by Hsu et al. [2020], we thus devise a synthesis process based on inter- and intra-streamline relationships to dynamically synthesize all the coherent streamlines with prescribed directions in parallel in preference to separately calculating integral curves step by step. Furthermore, because the topology of an unsteady flow may notably transform with time, similar to [Turk and Banks 1996], we incorporate a regulation mechanism with primitive operations to appropriately enhance each streamline's continuity and match the overall streamline coverage with the topology as time goes on.

As compared with conventional streamline integration, whose computation cost can significantly increase along with space-time dimensionality, through the synergy of the synthesis process and the regulation mechanism, our framework can possess the power to efficiently settle the entire streamline transitions that observe instantaneous flow topology without compromising output quality. In addition, as exemplified in Figure 1, uniform or spatially-varying streamline arrangements can be controllably formed according to specification (e.g. extra scalar inputs or local flow similarity) to render artistic compositions (e.g. stripe patterns) or emphasize flow characteristics (e.g. critical points). In virtue of further integrating the element field idea of Hsu et al. [2020] to facilitate controllability, our framework can autocomplete smooth streamline arrangements from partial inputs (i.e. control points/curves) in a one-step process. Here we implement the framework's prototype for application in pattern design and field visualization, including both 2D and 3D output domains, and exhibit our results via streamline animations.

2 Methodology

Given an output domain $\mathcal{D}$ with specific shape and a vector field $\mathcal{O}$ whose topological features can continually change in $\mathcal{D}$ over time t , our goal is to controllably arrange a set of coherent streamlines $\{\mathcal{S}\}$ tracing out $\mathcal{O}$ across $\mathcal{D}$ while suiting each streamline's continuity and the overall streamline coverage to the topology of $\mathcal{O}$ as t passes.

2.1 Streamline Representation

We approximate a single streamline $\mathcal{S}$ via a linkage of samples $\{s\}$. Each sample s has its own attributes, including a current position $\mathbf{p}$, a direction vector $\mathbf{o}$ derived from the normalized vector of $\mathcal{O}$ at $\mathbf{p}$, and a range radius r related to its weight. Like [Hsu et al. 2020], we can severally signify the samples' weights by 2D circles/3D spheres that can be used as bounding area/volume for streamline packing (see Figure 2 for a 2D example). In our synthesis process, we aim to best update $\mathbf{p}$ considering $\mathbf{o}$, while during the regulation mechanism, we regulate r resting on specification to vary the streamline density.

2.2 Streamline Synthesis

To well synthesize a series of streamlines with $\mathcal{O}$ frame by frame, we can repeatedly modulate both inter-streamline distribution and intra-streamline linkage with prescribed directions while relatively correlating the streamlines in adjoining time frames by means of each streamline's samples. Hence, we define a distribution E_d term, a linkage E_l term, and a coherence E_c term for particular objectives.

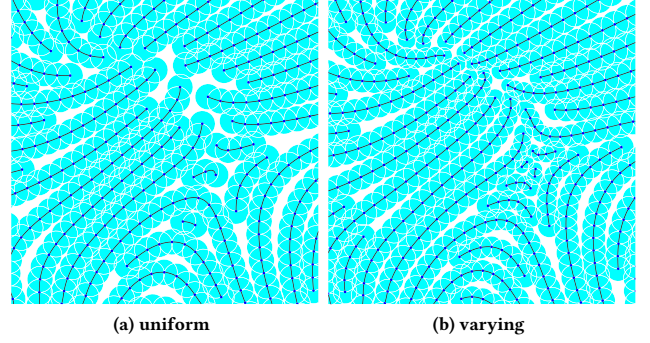


Figure 2: Streamline representation. We approximately depict streamlines (black curves) by a sparse array of linked samples (blue dots) with uniform (a)/varying (b) weights (cyan circles).

Distribution E_d . The core of well-distributed streamline creation, in general, is to space out all the streamlines with intended intervals in such a way that irregular streamline clustering can be avoided. For this end, we turn the streamline arrangement into the packing of samples with weights to regularize the inter-streamline distribution.

When our plan is to compactly aggregate a cluster of streamlines by their samples, regarding a streamline $\mathcal{S}$ composed of samples $\{s\}$, each sample s should retain a commensurate distance from each of its neighboring samples $\{s'\}$, coming from neighboring streamlines, in proportion to their weights. Since the minimum distance between the centers of two non-overlapping circles/spheres is equivalent to the sum of their range radii, for a pair of s and s' , a projection $\mathbf{proj}$ of s can be derived from the intersection node $\mathbf{p}$ of the segment among s and s' and their power bisector plus the necessitated displacement as shown in Figure 3, and the $\mathbf{proj}$ definition of s can be stated as:

$$\mathbf{proj}(s, s') = \mathbf{p}(s, s') + r(s) \frac{(\mathbf{p}(s) - \mathbf{p}(s'))}{|\mathbf{p}(s) - \mathbf{p}(s')|}, \quad (1)$$

$$\mathbf{p}(s, s') = \frac{\mathbf{p}(s) + \mathbf{p}(s')}{2} + \frac{(r(s'))^2 - (r(s))^2}{2 \cdot |\mathbf{p}(s) - \mathbf{p}(s')|} \frac{(\mathbf{p}(s) - \mathbf{p}(s'))}{|\mathbf{p}(s) - \mathbf{p}(s')|}.$$

With a weighted average of all pairs' projections of s , we can project the position of s that proportionally maintains the distance between each pair of s and s' for the inter-streamline distribution. Therefore, for $\mathcal{S}$ composed of $\{s\}$, the distribution E_d term can be defined as:

$$E_d(\mathcal{S}) = \sum_{s \in \mathcal{S}} \left| \mathbf{p}'(s) - \frac{\sum_{s' \in \mathcal{N}(s), s' \notin \mathcal{S}} (\mathbf{w}_d(s, s') \mathbf{proj}(s, s'))}{\sum_{s' \in \mathcal{N}(s), s' \notin \mathcal{S}} \mathbf{w}_d(s, s')} \right|^2, \quad (2)$$

$$\mathbf{w}_d(s, s') = \frac{1}{(\mathcal{R}(s, s'))^2}, \mathcal{R}(s, s') = \frac{|\mathbf{p}(s) - \mathbf{p}(s')|}{r(s) + r(s')},$$

where $\mathbf{p}'$ treated as a variable means a sample's destination position that we aim to resolve, $\mathcal{N}$ implies a sample's all neighboring samples $\{s'\}$ that can be established via a weighted Delaunay triangulation as in [Hsu et al. 2020], and $\mathbf{w}_d$ is an inverse square function used to calibrate the influence amid the paired samples depending on $\mathcal{R}$, which is the ratio that tells how far or close two circles/spheres are.

Linkage E_l . As each streamline's samples also have to be steadily linked in compliance with $\mathcal{O}$, for a tradeoff amid arithmetic accuracy and workload, we thereby introduce a bidirectional Euler integrator with a multilevel step size to manage the intra-streamline linkage.

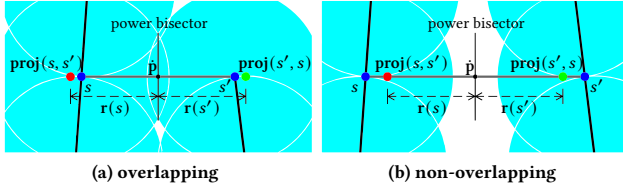


Figure 3: Streamline distribution. For a pair of overlapping (a) or non-overlapping (b) s and s' , we can reckon a projection of s (red dot) by moving from p with the required offset (i.e. $r(s)$) and analogously figure out a projection of s' (green dot), and the distance between the pair's projections equals the sum of the pair's weights (i.e. the least distance without overlap).

In terms of a streamline $\mathcal{S}$ comprising plural samples $\{s_1, \dots, s_m\}$, we rule the step size h amid two linked samples s_i and s_{i+1} by half of their weights' sum (i.e. $h(s_i, s_{i+1}) = \frac{r(s_i) + r(s_{i+1})}{2}$) as in Figure 2. Since the step size has a great impact on the precision of integration, for the reduction in numerical errors, we evenly divide the step size amid s_i and s_{i+1} into $(\kappa + 1)$ sub-step sizes via κ in-between samples as illustrated in Figure 4a, and the connection between each pair of linked samples within the streamline segment among s_i and s_{i+1} can be expressed in the manner of the forward Euler integrator as:

$$\begin{aligned} p(s_i^1) &= p(s_i) + \frac{h(s_i, s_{i+1})}{\kappa + 1} \mathbf{o}(s_i), \\ p(s_i^{j+1}) &= p(s_i^j) + \frac{h(s_i, s_{i+1})}{\kappa + 1} \mathbf{o}(s_i^j) \quad \text{for } 1 \leq j < \kappa, \\ p(s_{i+1}) &= p(s_i^\kappa) + \frac{h(s_i, s_{i+1})}{\kappa + 1} \mathbf{o}(s_i^\kappa), \end{aligned} \quad (3)$$

where s_i^j is the j -th in-between sample beginning forward with s_i . With Equation (3), the connection between s_i and s_{i+1} in a way that meets the forward Euler integrator can be succinctly expressed as:

$$p(s_{i+1}) = p(s_i) + \frac{h(s_i, s_{i+1})}{\kappa + 1} (\mathbf{o}(s_i) + \sum_{1 \leq j \leq \kappa} \mathbf{o}(s_i^j)). \quad (4)$$

By the same token, an expression of the connection between s_i and s_{i+1} abiding by the backward Euler integrator can be concluded as:

$$p(s_i) = p(s_{i+1}) - \frac{h(s_i, s_{i+1})}{\kappa + 1} (\mathbf{o}(s_{i+1}) + \sum_{1 \leq j \leq \kappa} \mathbf{o}(s_{i+1}^j)), \quad (5)$$

where s_{i+1}^j is the j -th in-between sample starting backward at s_{i+1} .

To ensure that the connection between s_i and s_{i+1} consistently complies with Equation (4) and (5) for the intra-streamline linkage, we can define the linkage E_l term for $\mathcal{S}$ comprising $\{s_1, \dots, s_m\}$ as:

$$\begin{aligned} E_l(\mathcal{S}) &= \sum_{1 \leq i < m} \left| p'(s_{i+1}) - p'(s_i) - \frac{h(s_i, s_{i+1})}{2 \cdot (\kappa + 1)} \partial(s_i, s_{i+1}) \right|^2, \\ \partial(s_i, s_{i+1}) &= \mathbf{o}(s_i) + \mathbf{o}(s_{i+1}) + \sum_{1 \leq j \leq \kappa} (\mathbf{o}(s_i^j) + \mathbf{o}(s_{i+1}^j)), \end{aligned} \quad (6)$$

where p' is the same as in Equation (2), $\mathbf{o}$ acquired at p in sequence is a constant, and κ can be tuned leaning on fidelity and performance. Like the graph similarity of Hsu et al. [2020], E_l can effectively guide a streamline's samples with the prescribed directions, and thanks to the bidirectional Euler integrator's symmetry, it can mutually counteract the integral deviations in some measure (see Figure 4b).

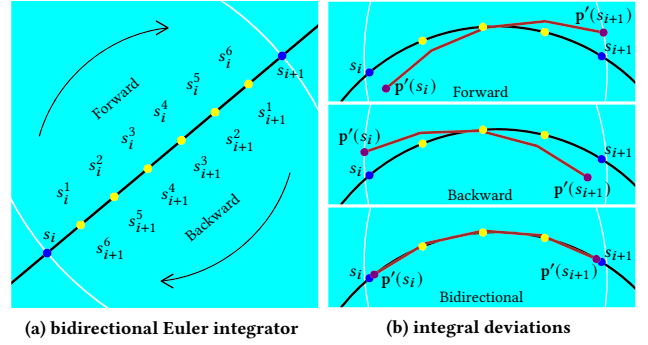


Figure 4: Streamline linkage. The streamline segment among s_i and s_{i+1} , evened out by κ in-between samples (yellow dots), can be built via the bidirectional Euler integrator (a) ($\kappa = 6$) in $(\kappa + 1)$ sub-step sizes. Due to the integral deviations (b) ($\kappa = 3$), the connection (brown polyline) between s_i and s_{i+1} obeying a single forward (up) or backward (middle) Euler integrator can result in the derived $p'(s_i)$ and $p'(s_{i+1})$ (purple dots) with notable inaccuracy, while the bidirectional one (down) can fairly offset the integral deviations for a preferable linkage.

Coherence E_c . To abate temporal aliasing or incoherence artifacts, we have to correspondingly relieve each streamline's displacement between consecutive time frames by restraining the offset of each of its samples as in [Jobard and Lefer 2000]. For a streamline $\mathcal{S}$ with samples $\{s\}$, the coherence E_c term can be consequently defined as:

$$E_c(\mathcal{S}) = \sum_{s \in \mathcal{S}} (w_c(s) |p'(s) - p(s)|^2), w_c(s) = 1 + \mathbf{o}(s) \cdot \mathbf{o}(s), \quad (7)$$

where p' is the same as in Equation (2), p and $\mathbf{o}$ treated as constants respectively denote a sample's last position and last direction vector in the previous time frame, and w_c is a confidence factor, namely the sample with similar direction vectors among time frames should offer a superior restraint. Note that for those newly inserted samples within each time frame (see Section 2.3), we rule out them from E_c .

2.3 Streamline Regulation

Since the topology of $\mathcal{O}$ can fluctuate over t , to reflect its conversion at every moment, we exploit primitive operations so as to duly adapt each streamline's continuity and the overall streamline coverage to the topology. Withal, with the usage of specification, we can globally manipulate the interspace between abutting streamlines such that desired streamline arrangements can be controllably brought about.

Primitive operations. As motivated by Turk and Banks [1996], we leverage primitive insertion, deletion, and combination operations to maneuver individual streamlines via samples. In brief, we insert new samples to elongate existing streamlines, delete overlapping samples to curtail overly jammed streamlines, and combine linkable samples originating from adjacent streamlines. As $\mathcal{R}$, used in E_d , can reveal whether two neighboring samples overlap, for a sample s surrounded with samples $\{s'\}$, we can determine whether s can be inserted or deleted by checking $\mathcal{R}$ for each pair of s and s' through a threshold ϵ . That is to say, if s is a new sample projected out from a streamline's extremity via E_l and has each pair's $\mathcal{R}$ larger than ϵ , we insert s to the extremity, or if s is an existing sample and there is

a pair's $\mathcal{R}$ smaller than ϵ , s is deleted from $\mathcal{D}$. For better streamline continuity and coverage, we utilize ϵ slightly lower than 1 to allow marginal overlaps between these paired samples (e.g. in Figure 2), and this is like making the interspace between abutting streamlines a bit less than the specified separating extent during the integration as in [Liu et al. 2006]. Likewise, for a pair of two endmost samples coming from two adjacent streamlines, we can check the pair's $\mathcal{R}$ with another threshold to determine whether to combine the pair.

Since empty areas may pop up between neighboring streamlines when the topology of $\mathcal{O}$ shifts, the insertion operation must capably populate new samples within such empty areas as well. Practically, we search for the paired non-overlapping samples (e.g. in Figure 3b) with the maximum $\mathcal{R}$ larger than 2 for two neighboring streamlines, and if the pair exists, we determine whether to insert a new sample s at the pair's $\hat{\mathbf{p}}$ by checking $\mathcal{R}$ for s as described above. Once inserted, we treat s as a stream-point (i.e. a special case of a streamline with only one sample) and then elongate it from its both ends if possible. On this basis, we can repetitively execute the insertion operation to fill the empty areas with samples until no space in $\mathcal{D}$ is available for any sample insertion. Moreover, by adding extra boundary samples, we can similarly tackle the empty areas near the domain boundary.

Specification. The streamline density can be spatially varied by singly setting each sample's weight to its specified scalar originating from extra scalar inputs (e.g. a procedure function, a scalar field, or user-designated scalars), local flow similarity, or a blend amid them. In order that each sample's weight adheres to the specification and smoothly alters among neighboring samples, for a streamline $\mathcal{S}$ made up of samples $\{s\}$, below we refer to [Hsu et al. 2020] to define a regulation E_r term, exemplified by the local flow similarity σ , as:

$$E_r(\mathcal{S}) = \sum_{s \in \mathcal{S}} (|\mathbf{r}'(s) - \sigma(s)\mathbf{c}(s)|^2 + \sum_{s' \in \mathcal{N}(s)} (\mathbf{w}_d(s, s') |\mathbf{r}'(s) - \mathbf{r}'(s')|^2)), \quad (8)$$

$$\sigma(s) = \max \left\{ \left(\frac{\sum_{s' \in \mathcal{N}(s)} (\mathbf{w}_d(s, s') (1 + \mathbf{o}(s) \cdot \mathbf{o}(s'))) }{2 \cdot \sum_{s' \in \mathcal{N}(s)} \mathbf{w}_d(s, s')} \right)^\eta, \kappa \right\},$$

where $\mathbf{r}'$ treated as a variable indicates a sample's destination weight that we aim to resolve, $\mathbf{c}$ stands for the maximum range radius for a sample's weight, σ is gauged by weighing the cosine similarity of direction vectors for a sample and each of its neighboring samples in pairs, η with a default of 1 is a tunable parameter used to modify the effect of σ , and κ manifesting the lower bound of σ is set to 0.5.

When only considering a fixed $\mathbf{c}$ without σ in E_r (as in Figure 2a), all the samples hold the same weight. In contrast, when with σ in E_r (as in Figure 2b), the samples with larger weights can appear over smooth zones, whereas the ones with smaller weights collectively come out around critical points/separatrices. Through freely mixing or replacing σ or $\mathbf{c}$ in E_r with the aforementioned specifications, E_r can be specialized for various application demands (e.g. in Figure 1).

3 Solver

We synergize the synthesis process and the regulation mechanism by iterations to create well-distributed streamlines. In each iteration, we first sort out all the streamlines via the primitive operations, followed by solving E_r via linear least squares if the specification is assigned. Subsequently, E_d , E_l , and E_c can also be solved together via linear least squares. For the output, each streamline is shaped from its samples by the cubic Hermite interpolation [Liu et al. 2006].

4 Application

The proposed framework can favor streamline-driven applications under time-varying scenarios, especially those unexplored in 3D yet. At the moment, we apply it to pattern design and field visualization.

Pattern design. As demonstrated in Figure 1a, our framework can integrate with the element field idea of Hsu et al. [2020] to design time-varying patterns from partial inputs (i.e. control points/curves) represented by samples with specified direction vectors and weights (i.e. extra scalar inputs). With [Hsu et al. 2020], at each time frame, each sample's direction vector and weight of all streamlines can be wholly derived from the partial inputs' samples treated as constants. To enrich the resulting patterns' visual diversity, we can optionally impart the output streamlines with assorted styles of appearance, such as a cyclic texture [Jobard and Lefer 2000] or a tapering effect [Turk and Banks 1996], in post-processing. As manually specifying control points/curves may need heavy effort and expertise, we plan to develop a user interface that can streamline the design workflow.

Field visualization. Our framework can opportunely characterize unsteady flows by streamlines, covering 3D cases (e.g. in Figure 1b), to help understand how their topological features evolve in time. Additionally, Figure 5 displays that we can directly associate $\mathbf{c}$ in E_r with the viewport's position for multi-resolution field visualization. For more generalization, we envision to extend the framework to the visualization of time-dependent tensor fields as our future work.

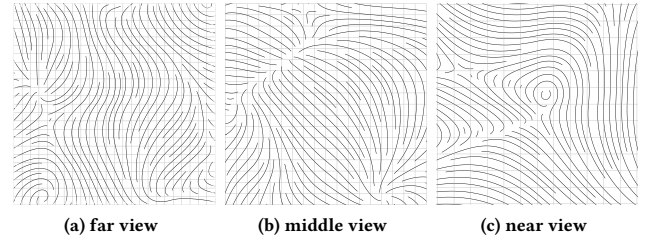


Figure 5: Multi-resolution field visualization. We can zoom in from far (a) to middle (b) to near (c) views over t while keeping the same streamline density in the viewport by accordingly scaling down $\mathbf{c}$ in E_r with reference to the viewport's position.

Acknowledgments

The first author sincerely thanks his parents for this study's support.

References

- Chen-Yuan Hsu, Li-Yi Wei, Lihua You, and Jian Jun Zhang. 2020. Autocomplete Element Fields. In *CHI '20*. 1–13. <https://doi.org/10.1145/3313831.3376248>
- Bruno Jobard and Wilfrid Lefer. 2000. Unsteady Flow Visualization by Animating Evenly-Spaced Streamlines. *Comput. Graph. Forum* 19, 3 (2000), 31–39. <https://doi.org/10.1111/1467-8659.00395>
- Zhanping Liu, Robert J. Moorhead II, and Joe Groner. 2006. An Advanced Evenly-Spaced Streamline Placement Algorithm. *IEEE Transactions on Visualization and Computer Graphics* 12, 5 (Sept. 2006), 965–972. <https://doi.org/10.1109/TVCG.2006.116>
- Tony McLoughlin, Robert S. Laramée, Ronald Peikert, Frits H. Post, and Min Chen. 2010. Over Two Decades of Integration-Based, Geometric Flow Visualization. *Comput. Graph. Forum* 29, 6 (2010), 1807–1829. <https://doi.org/10.1111/j.1467-8659.2010.01650.x>
- Sudhanshu Sane, Roxana Bujack, Christoph Garth, and Hank Childs. 2020. A Survey of Seed Placement and Streamline Selection Techniques. *Comput. Graph. Forum* 39, 3 (2020), 785–809. <https://doi.org/10.1111/cgf.14036>
- Greg Turk and David Banks. 1996. Image-Guided Streamline Placement. In *SIGGRAPH '96*. 453–460. <https://doi.org/10.1145/237170.237285>